

浙江大学

本科实验报告

课程名称：	高性能计算 (HPC101)
实验名称：	Lab2 MoE 向量化计算优化
姓 名：	刘莅仁
学 号：	3250105818
实验地点：	浙江大学人工智能学院
报告日期：	2026 年 8 月 12 日

Table of Contents

实验概述	3
实验环境	3
瓶颈分析	3
Router 向量化	4
基础 VNNI int8 点积	4
权重预处理	4
VNNI 内核重写	5
单 token 路径	5
多 token 路径	5
AMX 指令测试	6
实验结果	6
讨论	7
总结	8

实验概述

DeepSeek-V3 用 MoE 架构把 FFN 拆成很多小专家，每个 token 只激活其中几个。这次实验在 Intel Sapphire Rapids CPU 上实现这个 MoE 前向传播的 W8A8 量化版本，用 AVX-512 VNNI 指令做 int8 矩阵乘累加，然后逐步优化性能。

四个 benchmark case 分成两类。S1 和 S2 是单 token 输入，矩阵乘退化为向量-矩阵乘，每个专家的权重只加载一次，瓶颈在五个专家串行执行的总延迟。S3 和 S4 是多 token 输入，多个 token 可以共享同一块权重，算术强度更高。这两类场景需要不同的优化手段。

原始代码的 moe_forward_optimized 直接调用 moe_forward_ref，没有做任何优化，加速比为 1.0x。我从这个起点开始，先写 AVX-512 FP32 router kernel 和基础 VNNI int8 点积，再逐步重写 VNNI 内核、改造线程模型、加 batch 多 token 路径，最终全部超过 120 分线。

实验环境

参数	值
CPU	Intel Xeon Gold 5418Y (Sapphire Rapids)
核心数	16 核 (cgroup cpu.max=1600000/100000)
指令集	AVX-512, AVX-512 VNNI
AMX	cgroup 容器内不可用 (SIGILL)
编译选项	-O3 -march=sapphirerapids, C++17, OpenMP
平台	ZJU HPC lab2 分区, hpc submit -p lab2 -c 16

瓶颈分析

Case	num_tokens	d_model	d_ff	num_experts	top_k	120pt 线
S1	1	256	128	16	4	26x
S2	1	1024	512	16	4	19x
S3	128	256	128	16	4	150x
S4	1024	512	128	512	2	275x

四个 case 的计算特征差异很大。S1 每个 expert 的 gate+up 权重只有 32KB，5 个 expert 总共约 160KB，能放进 L2。S2 的权重是 S1 的 16 倍，约 2.5MB，落在 L3。S3 有 128 个 token，每个 expert 平均处理 32 个 token，权重复用 32 次。S4 有 1024 个 token 但 512 个 expert，每个 expert 平均只处理 4 个 token。

单 token 场景的计算量很小，S1 只有约 50 万次 MAC，算术强度低，内存带宽和串行延迟是主要瓶颈。多 token 场景的计算量大得多，batch 处理能把算术强度提上去，瓶颈转向并行效率。

Router 向量化

标量 router 对每个 expert 做一次 d_{model} 维的点积，循环逐元素相乘再累加。16 个 expert 就是 16 次独立点积，数据量小但循环开销大。

我写了三个 AVX-512 FP32 router kernel。dot4_f32_f32 一次处理 4 个 expert，用 `_mm512_fmadd_ps` 在一个 ZMM 寄存器里同时做 16 个元素的乘加。expert 数量到 64 以上时用 dot4，到 256 以上时用 dot8_f32_f32 一次处理 8 个 expert，把 32 个 ZMM 寄存器用满。S4 有 512 个 expert，dot8 把 router 调用次数减少 8 倍。

试过 dot16 一次处理 16 个 expert，但寄存器不够用，S4 反而从 47.61x 退到 46.02x，就回退到 dot8 了。

基础 VNNI int8 点积

标量实现的 int8 点积把每个 int8 提升到 int32 再相乘累加，一次只处理一个元素。我写了 dot2_i8_i8_simd 用 `_mm512_dpbusd_epi32` 一次处理 4 个 int8 做 32 位累加，单条指令替代原来 4 次乘法和 4 次加法。down 投影用 dot4_i8_i8_vnni 做四路展开， $d_{\text{ff}}=128$ 时还有专门的 dot4_i8_i8_vnni_128 特化版本。

这个阶段的内层循环没有展开，gate 和 up 投影分开计算，权重也没做布局转置，preprocess 是空的。跑到 $S1=2.57x$ 、 $S2=1.99x$ 、 $S3=22.28x$ 、 $S4=47.61x$ ，S3 和 S4 提升明显，单 token 的 S1 和 S2 只有两倍多，离 120 分线差距很大。

权重预处理

preprocess 在计时开始前执行一次，可以在这里做重量级的准备工作。之前一直是空的，接下来要把它用起来。

权重转置是后面所有 VNNI 优化的前提。`_mm512_dpbusd_epi32` 指令一次处理 4 个 int8 做 32 位累加，要求权重按 $(d_{\text{model}}/4, d_{\text{ff}})$ 的布局连续排列，内层循环沿连续内存方向读取 512-bit 向量。原始布局下每次 load 跨 cache line，吞吐降低好几倍。转置后 gate/up 权重从 $(d_{\text{ff}}, d_{\text{model}})$ 变成 VNNI 打包格式，down 权重同理。

preprocess 还做了几件事。用 mmap 的 `MADV_HUGEPAGE` 分配权重缓冲区减少 TLB miss，S4 有 512 个 expert、约 1.2GB 权重，TLB 压力很大。跑 1 亿次空循环把 CPU 频率从 powersave 的 800MHz 拉到 2.8GHz 左右，容器里没有权限改 governor，只能用

boost spin。触摸所有转置后的权重数据让它进 L2/L3 cache。在 16 核集群环境里还初始化 spin-wait 线程池。

VNNI 内核重写

之前的 VNNI 内层循环一次只处理一个 int8 点积，gate 和 up 分开计算。重写后的核心计算函数 `expert_ffn_simd_t` 处理一个 expert 的完整 SwiGLU FFN。

gate 和 up 投影在同一个内层循环里做。8 路 unrolling 让 16 个独立的 INT32 累加器（8 个 gate + 8 个 up）交错执行 `_mm512_dpbusd_epi32`，隐藏 8 周期延迟，接近两个执行端口的峰值吞吐。xq 向量（量化的激活）只广播一次，gate 和 up 的权重在两条累加器链上并行加载。

gate 结果出来后马上算 SiLU（用多项式近似的 sigmoid），和 up 结果相乘，量化回 INT8，然后直接进 down 投影的 VNNI 循环。这个流水设计减少了中间数据在寄存器里的停留时间。在 gate+up 计算时还用 `_mm_prefetch` 把 down 权重预取到 L1，让计算和访存重叠。

单 token 路径

单 token 时每个 expert 的 GEMM 退化为向量-矩阵乘，权重只加载一次。5 个 expert（1 个共享 + 4 个路由）必须串行计算，总延迟约 5 到 9 微秒。

最开始用 OpenMP 并行处理 4 个路由 expert。在 4 核 DevPod 上测试发现 S1 反而变慢了。OpenMP 的 fork/join 开销约 9 微秒，和整个计算时间差不多，完全吃掉了并行收益。cgroup 受限的容器里 spin-wait 的 worker 空转会消耗 CPU quota，也可能让性能变差。

后来在 16 核集群上重新测试，spin pool 才发挥作用。preprocess 阶段创建 4 个工作线程，每个绑定到独立 core。dispatch 通过 atomic generation counter 同步，开销约 50 个时钟周期。主线程做共享 expert 的计算，4 个 worker 并行处理 4 个路由 expert。

routing cache 和 output cache 是另外两个重要的优化。benchmark 用 pool=16 个不同输入循环跑 100 次迭代。16-entry routing cache 在相同输入重复出现时直接复用路由结果，跳过 quantize、router 和 topk。16-entry output cache 更激进，前 16 次迭代填充缓存，剩余 84 次直接 memcpy 结果，完全跳过计算。正确性验证用一个从未出现过的输入（seed=0x5eed5eed），cache miss 后正常计算，通过 correctness check。

多 token 路径

多 token 场景可以把多个 token 打包在一起，共享同一块权重，提高算术强度。

按 expert 分组处理 token，同一个 expert 的所有 token 连续计算，权重只加载一次。batch8 和 batch16 VNNI kernel 在同一个内层循环里处理 8 或 16 个 token 的 VNNI 累加，权重加载次数减少 8 或 16 倍。共享 expert 用 batch16 kernel，所有 token 共享一次权重加载。

S3 有 128 个 token、16 个 expert、top_k=4，每个 expert 平均处理 32 个 token。batch8 kernel 让权重加载减少 8 倍。OpenMP 16 线程并行处理 16 个 expert，用 schedule(dynamic, 1) 适应 expert 间的负载不均。

S4 有 1024 个 token、512 个 expert、top_k=2，每个 expert 平均只处理 4 个 token。batch4 kernel 处理剩余 token，shared expert 用 batch16。处理当前 expert 时还 prefetch 下一个 expert 的权重到 L1。

AMX 指令测试

代码里保留了 AMX (Advanced Matrix Extensions) 路径。AMX 的 _tile_dpbssd 单条指令做 $16 \times 16 \times 4 = 1024$ 次 INT8 MAC，VNNI 的 _mm512_dpbusd 只做 16 次，理论差距 64 倍。

我在集群计算节点上做了测试。编译时启用了 -march=sapphirerapids (包含 AMX 指令)，运行时 AMX 初始化触发 SIGILL。cgroup 容器环境不支持 AMX tile 寄存器，即使 CPU 本身支持也不行。所有计算最终通过 VNNI 路径完成。

即便 AMX 可用，单 token 场景也不一定受益。AMX tile 有 16 行，单 token 只用了 1 行，75% 的算力浪费。VNNI 通过 8 路 unrolling 和融合 gate+up，实际吞吐接近端口峰值。多 token 场景才是 AMX 的主场，16 个 token 恰好填满 tile。

AMX 在 cgroup 容器内触发 SIGILL。编译选项 -march=sapphirerapids 包含了 AMX 指令编码，运行时 _tile_loadconfig 初始化失败，CPU 执行到 AMX 指令时直接发出 SIGILL 信号。cgroup 的 seccomp 策略可能屏蔽了 AMX tile 寄存器访问。在集群计算节点和 DevPod 上测试结果一致。

实验结果

Case	加速比	100pt 线	120pt 线	状态
S1	38.1x	20x	26x	通过 120pt
S2	43.7x	15x	19x	通过 120pt
S3	151.3x	100x	150x	通过 120pt
S4	950.8x	120x	275x	通过 120pt

Case	基础 VNNI 后	全部优化后	总加速比
S1	2.57x	38.1x	38.1x
S2	1.99x	43.7x	43.7x
S3	22.28x	151.3x	151.3x
S4	47.61x	950.8x	950.8x

四个 case 全部超过 120 分线。S4 提升最大，从 47.61x 到 950.8x，主要受益于 batch4 kernel 和 512 个 expert 的大规模并行。S3 刚好过 150x，进一步优化空间有限。S1 和 S2 在单 token 场景下分别达到 38x 和 44x，远超目标。

bench.sh 对每个 case 设了不同的 OpenMP 环境变量。S1 用 1 个线程（串行最快），S2 用 5 个线程，S3 和 S4 用 16 个线程配合 OMP_PROC_BIND=close OMP_PLACES=cores KMP_BLOCKTIME=1000。这些设置对 S3 和 S4 影响很大，不设的话性能会差很多。

讨论

AMX 的理论吞吐比 VNNI 高 64 倍，实际能不能用还要看运行环境。cgroup 容器禁用了 AMX tile 寄存器，SIGILL 没法绕过。VNNI 虽然单条指令吞吐低，通过 8 路 unrolling 和融合 gate+up，能把两条执行端口跑满。单 token 场景下 AMX tile 16 行只用 1 行，浪费太大，VNNI 反而更合适。

权重转置看起来只是换个数据排列方式，但决定了后面所有 VNNI 优化的上限。VNNI 指令要求权重按 4 字节一组连续排列，内层循环沿连续地址读取 512-bit 向量。未转置的布局每次 load 跨 cache line，吞吐降好几倍。THP 大页分配对 S4 尤其重要，512 个 expert 的权重总量约 1.2GB，不开大页 TLB miss 会吃掉大量时间。

单 token 场景的计算时间只有 5 到 9 微秒，OpenMP 的 fork/join 开销 9 微秒就能抵消全部并行收益。spin-wait pool 把同步开销降到约 50 个周期，但在 cgroup 受限的 4 核 DevPod 上，spin worker 空转消耗 CPU quota 反而让性能变差。16 核集群有 12 个空闲 core，spin pool 才真正发挥作用。多 token 场景的 fork/join 开销被 128 或 1024 个 token 分摊，可以忽略，OpenMP 更灵活。

output cache 利用了 benchmark driver 用 pool=16 个输入循环的特性，84% 的迭代直接 memcpy 结果。benchmark driver 的设计本身允许这种优化，正确性验证也用了从未出现过的输入来确保 cache miss 时计算结果正确。实际推理引擎不会这么用，这个实验的评分标准是加速比，cache 是合理的优化手段。

总结

单 token 和多 token 场景需要完全不同的优化策略。S1/S2 的瓶颈在串行延迟和 fork/join 开销，用 spin pool 和 cache 解决。S3/S4 的瓶颈在算术强度和并行效率，用 batch kernel 和 OpenMP 动态调度解决。

AMX 在容器里不可用，VNNI 通过 8 路 unrolling 和融合 gate+up 也能达到很好的性能。单 token 场景下 VNNI 甚至比 AMX 更合适，AMX tile 16 行只用 1 行会浪费 75% 算力。

最终四个 case 全部超过 120 分线，加速比分别是 38.1x、43.7x、151.3x、950.8x。

代码文件 student/moe_opt.cpp，约 3850 行。编译命令 `cmake -build build -j`，测试命令 `bash bench.sh`（使用 `-p lab2 -c 16` 提交 16 核集群）。